Dedeepya Majety: PwC Acceleration Center Technical & Architectural Interview Dossier

Target: PwC Acceleration Center India — FS - Data Engineer - Associate (Req ID: 4467378631)

Candidate: Dedeepya Majety
Current Role: Associate - Data Engineer @ Cognizant (3.9 YOE)
Target Practice: Financial Services (FS) Advisory & Acceleration Centers
Location: Bengaluru East, Karnataka, India
Notice Period Status: 🟢 Serving Official Notice Period · Available in 30 Days
Compensation Floor Anchor: Confirmed Offer @ 10.9 LPA · **Target Range:** 15.0 – 18.0+ LPA
Verified Credentials: Databricks Certified Data Engineer Associate, Microsoft Certified PL-300 (Power BI), Microsoft Certified PL-900

---

1. Executive Positioning & Recruiter/HM Opening Hook

1.1 The 60-Second Verbal Opening Hook (Memorize & Deliver Naturally)

> *"I am an Azure Data Engineer with nearly 4 years of experience at Cognizant, specializing in building high-trust, audited financial and enterprise data platforms using Azure Databricks, PySpark, and Delta Lake.*
>
> *In my current delivery at Cognizant, I contributed to modernizing 40+ legacy ETL workflows to an Azure Databricks Medallion Lakehouse, cutting batch runtimes by 40% and lowering compute spend by ~30%. Because Financial Services platforms require absolute data integrity, auditability, and zero reconciliation drift, I engineered automated row-count and checksum verification frameworks, implemented fine-grained governance using Databricks Unity Catalog, and built conformed dimensional models powering executive reporting.*
>
> *In PwC's JD, you explicitly note that expertise in **Unity Catalog, Delta Lake, and automated workflows** is what 'sets a candidate apart'—all of which represent my daily production stack. Furthermore, I hold both the Databricks Certified Data Engineer Associate and Microsoft Power BI PL-300 certifications. I am actively serving my notice period with an offer in hand at ~11 LPA and am available to join PwC within 30 days."*

---

2. PwC Financial Services Practice Context & Alignment

PwC FS Practice Dimension	PwC Requirement & Context	Dedeepya Production Proof Point (Cognizant)
Financial Data Integrity	Zero data loss, exact ledger reconciliation	Automated row-count, hash checksum, and balance reconciliation scripts verifying 100% data parity.
What Sets You Apart	Unity Catalog, Databricks workflows, Delta Lake	Production deployment of Unity Catalog 3-level namespace, column masking, and modular Apache Airflow DAGs.
Data Modeling & BI	Analytical profiling, reporting datasets	Star-schema modeling in Gold Delta tables; Microsoft Certified PL-300 Power BI Data Analyst.
Audit & Governance	Regulatory traceability and access controls	Informatica MDM entity deduplication (~15% duplicates removed) and Delta Lake Time Travel audit logs.

---

3. Top 10 Technical Viva & Architecture Defense Questions for PwC FS

Q1: Financial Services data pipelines require 100% auditability and data reconciliation. How do you design an automated reconciliation framework in PySpark?

Structured Viva Answer: - **Reconciliation Invariants:** In Financial Services, row counts alone are insufficient; you must verify volume, checksums, and business financial aggregates. - **Three-Tier Verification Architecture:** 1. **Volume Reconciliation:** Verify `count(source) == count(target_inserted) + count(target_updated) + count(quarantined_corrupt)`. 2. **Checksum Parity:** Concatenate all business critical columns (e.g., Account ID, Transaction Amount, Currency, Trade Date), compute SHA-256 hash (`F.sha2(F.concat_ws("||", *cols), 256)`), and verify set equivalence using anti-joins. 3. **Financial Aggregate Reconciliation:** Compute control totals by transaction currency and date: `sum(debits)`, `sum(credits)`, and `net_balance`. Compare source control totals to target Gold mart control totals down to 4 decimal places. - **Automated Alerting:** If reconciliation variance exceeds 0.00%, the Airflow DAG immediately raises an alert, blocks downstream Gold table promotion, and writes mismatched records to a quarantine table.

Q2: How do you implement Slowly Changing Dimensions (SCD Type 2) in Delta Lake for financial accounts?

Structured Viva Answer: - **Business Need:** Track historical changes to customer credit limits, risk ratings, or compliance status over time with `start_date`, `end_date`, and `is_current_flag`. - **Implementation Mechanism:** `sql -- Stage 1: Identify updated records by joining incoming feed with current dimension records -- Stage 2: Merge statement with unioned dataset to expire old rows and insert new versions MERGE INTO gold.dim_financial_account AS target USING (-- Rows to insert (new accounts + new versions of updated accounts) SELECT staged.account_id, staged.credit_limit, staged.risk_rating, staged.effective_date AS start_date, NULL AS end_date, true AS is_current FROM staged_accounts staged UNION ALL -- Rows to close out (set is_current = false) SELECT staged.account_id, staged.credit_limit, staged.risk_rating, target.start_date, staged.effective_date AS end_date, false AS is_current FROM staged_accounts staged JOIN gold.dim_financial_account target ON staged.account_id = target.account_id WHERE target.is_current = true AND (staged.risk_rating <> target.risk_rating OR staged.credit_limit <> target.credit_limit)) AS source ON target.account_id = source.account_id AND target.start_date = source.start_date WHEN MATCHED THEN UPDATE SET target.end_date = source.end_date, target.is_current = source.is_current WHEN NOT MATCHED THEN INSERT (account_id, credit_limit, risk_rating, start_date, end_date, is_current) VALUES (source.account_id, source.credit_limit, source.risk_rating, source.start_date, source.end_date, source.is_current);`

Q3: PwC explicitly looks for Unity Catalog expertise. How does Unity Catalog support financial regulatory compliance (e.g., SOX, GDPR, BCBS 239)?

Structured Viva Answer: - **Automated Audit Logging:** Unity Catalog records every access event, schema alteration, and table query into system tables (`system.access.audit`). Auditors can run SQL queries to see exactly who accessed, modified, or exported customer financial data over the past 365+ days. - **Cryptographic Lineage:** Lineage is automatically captured down to the column level. If an executive balance sheet number is questioned, Unity Catalog displays the exact lineage DAG back through Gold, Silver, Bronze, and raw landing files. - **Dynamic Masking for PII/PCI Data:** Cardholder data, bank account numbers, and tax identifiers are automatically masked based on the user's Active Directory / Entra ID role membership without duplicating physical tables. - **Immutable Storage:** Underlying Delta files are stored in write-once, tamper-evident ADLS Gen2 storage accounts with immutable blob policies.

Q4: How do you handle Late-Arriving Data in financial transaction pipelines?

Structured Viva Answer: - **The Challenge:** A trade or credit transaction that occurred on October 1st arrives in the lakehouse on October 14th due to a weekend clearing house lag. - **Solution Strategy:** 1. **Decouple Event Time from Ingestion Time:** Ingest into Bronze with `ingestion_timestamp = current_timestamp()`, while preserving `transaction_timestamp` as the business event time. 2. **Silver Partitioning:** Partition Silver and Gold tables by

`transaction_date` (event date), not ingestion date. 3. **Idempotent Merge:** When the late-arriving batch processes, Delta `MERGE INTO` updates the historical partition (`transaction_date = '2026-10-01'`) without re-processing unrelated dates. 4. **Downstream Trigger:** Emit an event or Airflow trigger to re-compute aggregate balances for all dates from October 1st forward, ensuring downstream financial reports reflect true historical balances.

Q5: How does Delta Lake Time Travel support financial audits and regulatory inquiries?

Structured Viva Answer: - **Audit Scenario:** An auditor asks: *"What was the exact reported financial position of this customer segment as of March 31st at 11:59 PM?"* - **Solution:** Delta Lake stores the entire commit history in `_delta_log/` . - **Query Syntax:** ```sql -- Time travel via timestamp SELECT * FROM gold.customer_positions TIMESTAMP AS OF '2026-03-31 23:59:00';

-- Time travel via commit version SELECT * FROM gold.customer_positions VERSION AS OF 42; `- **Disaster Recovery:** If an incorrect ETL job corrupted Silver data at 2 AM, the table can be rolled back to its known good state in seconds:` sql RESTORE TABLE silver.transactions TO VERSION AS OF 41; ```

Q6: How do you design Power BI semantic models over Databricks for sub-second financial reporting? (PL-300 Alignment)

Structured Viva Answer: - **Architecture:** Gold Delta tables in ADLS Gen2 connected to Databricks SQL Serverless or Microsoft Fabric OneLake. - **Design Principles:** - **Star Schema:** Enforce 1:Many relationships from Dimension tables to Fact tables; avoid snowflake hierarchies or many-to-many relationships. - **Storage Mode Selection:** - **Import Mode (Aggregated):** For monthly C-suite P&L dashboards requiring sub-second slice-and-dice interaction. - **Direct Lake / DirectQuery (Detailed):** For drill-through to individual transaction-level rows without waiting for multi-hour dataset refreshes. - **Pre-Aggregated Summary Tables:** Build daily and monthly snapshot tables in the Gold layer so Power BI calculates fewer runtime DAX measures over hundreds of millions of rows.

Q7: What is the difference between structured streaming and batch processing in Databricks, and when do you use each in financial platforms?

Structured Viva Answer: - **Batch Processing:** - Scheduled executions (e.g., hourly, daily closing). - High throughput, cost-efficient, optimal for heavy historical aggregations, complex multi-table joins, and regulatory end-of-day batch reconciliation. - **Structured Streaming (with Delta Lake):** - Continuous or micro-batch processing triggered via `.trigger(availableNow=True)` or `.trigger(processingTime="1 minute")` . - Exactly-once processing semantics guaranteed by Delta Lake checkpoints and write-ahead logs. - Best for real-time fraud detection alerts, payment gateway status monitoring, and low-latency intraday position updates. - **Dedeepya's Production Stance:** In Cognizant's modernization, we used Auto Loader (`cloudFiles`) with `.trigger(availableNow=True)` to get the scalability of streaming with the cost predictability of scheduled batch jobs.

Q8: How did you handle Master Data entity resolution and duplicate records across enterprise systems?

Structured Viva Answer: - **Situation:** Enterprise client acquisitions introduced multiple overlapping CRM and ERP systems with conflicting customer entity records. - **Action:** - Built match-and-merge pipelines in Informatica MDM and conformed Silver layers in Databricks. - Defined deterministic match rules (Tax ID, Legal Entity Identifier) and probabilistic fuzzy matching (Name + Address phonetics). - Configured source system trust scores and attribute survivorship rules to establish single Golden Records. - **Result:** Eliminated ~15% duplicate entities across the enterprise catalog, preventing duplicate billing and ensuring unified regulatory reporting.

Q9: How do you handle schema drift in incoming vendor files without crashing financial pipelines?

Structured Viva Answer: - **Auto Loader Schema Rescue:** In Databricks Auto Loader, configure `.option("cloudFiles.schemaLocation", "/checkpoints/schema")` . - **Rescued Data Column:** Auto Loader automatically detects new or unparseable columns and writes them to the `_rescued_data` column rather than

failing the job. - **Automated Quarantine:** Silver PySpark pipelines inspect `_rescued_data`. If unparseable records exist, valid transactions are loaded into Silver while anomalous records are diverted to a quarantine table with an automated Airflow alert.

Q10: How do you optimize costs when running Databricks workloads for enterprise clients?

Structured Viva Answer: - **Automated Job Clusters:** Run production pipelines on automated Job Clusters (1 DBU cost) instead of Interactive All-Purpose Clusters (3x cost). - **Auto-Termination & Right-Sizing:** Set cluster auto-termination to 15 minutes and configure auto-scaling worker nodes (e.g., min 2, max 6 nodes) based on task backlog. - **Photon Engine:** Enable Photon for vectorized C++ execution on heavy Gold aggregation queries. - **Delta Optimization:** Regularly run `OPTIMIZE` and `VACUUM` to manage small file sprawl and trim obsolete storage.

---

4. Live Coding Defense Drills (PySpark & SQL)

Drill 1: Financial Balance Reconciliation Check

```
import pyspark.sql.functions as F

# Compute control totals between Source and Gold Target
source_summary = (
    source_df
    .groupBy("currency", "transaction_date")
    .agg(
        F.count("*").alias("src_count"),
        F.sum("amount").alias("src_total_amount")
    )
)

gold_summary = (
    gold_df
    .groupBy("currency", "transaction_date")
    .agg(
        F.count("*").alias("tgt_count"),
        F.sum("amount").alias("tgt_total_amount")
    )
)

# Join and detect any discrepancy
reconciliation_df = (
    source_summary
    .join(gold_summary, on=["currency", "transaction_date"], how="outer")
    .withColumn("count_diff", F.col("src_count") - F.col("tgt_count"))
    .withColumn("amount_diff", F.round(F.col("src_total_amount") - F.col("tgt_total_amount"), 4))
    .filter((F.col("count_diff") != 0) | (F.col("amount_diff") != 0.0000))
)

assert reconciliation_df.count() == 0, "Critical Financial Reconciliation Mismatch Detected!"
```

Drill 2: Rolling 30-Day Moving Average in PySpark

```
from pyspark.sql.window import Window
import pyspark.sql.functions as F

# Calculate 30-day rolling transaction volume per account
window_spec = (
    Window
    .partitionBy("account_id")
    .orderBy(F.col("transaction_date").cast("timestamp").cast("long"))
    .rangeBetween(-30 * 86400, 0)
)

rolling_df = (
    transactions_df
    .withColumn("rolling_30d_amount", F.sum("amount").over(window_spec))
    .withColumn("rolling_30d_count", F.count("*").over(window_spec))
)
```

---

5. Reverse Inversion Questions to Ask PwC AC Interviewers

1. *"How is the Financial Services practice at PwC Acceleration Center balancing Databricks Lakehouse with Microsoft Fabric for clients with existing Power BI footprints?"*
2. *"What regulatory compliance frameworks (e.g., BCBS 239, SOX, GDPR) most heavily dictate the architectural patterns and audit requirements in your current client engagements?"*
3. *"How does the team handle cross-border data residency constraints when delivering solutions from PwC Acceleration Centers for US and European financial clients?"*
4. *"What is the deployment and testing lifecycle for PySpark pipelines and Unity Catalog governance rules within your advisory engagements?"*

---

6. Compensation & Notice Period Strategy for PwC AC

- **Current Status:** Officially serving 30-day notice period at Cognizant.
- **Anchor Offer:** 10.9 LPA (Infosys confirmed).
- **Target PwC Compensation:** 15.0 – 18.0 LPA.
- **Script:** *"I am currently serving my official notice period at Cognizant and can join PwC within 30 days. I have already secured an offer at ~11 LPA. Because this role directly leverages my production background in Databricks, Unity Catalog, and Delta Lake along with my Databricks and Power BI certifications, I am targeting between 16 and 17.5 LPA."*